Agent Farm — Görev, Oturum ve Yeniden Deneme (Retry) Mimarisi

Kapsamlı Mühendislik ve Kullanıcı Başvuru Kılavuzu

Modül Referansı: `vsc_AgentFarm/src/core/tasks.ts` • `retry.ts` • `media/tasksPage.js` • Versiyon: 2.5

1. Giriş ve Sistemin Genel Mimarisi

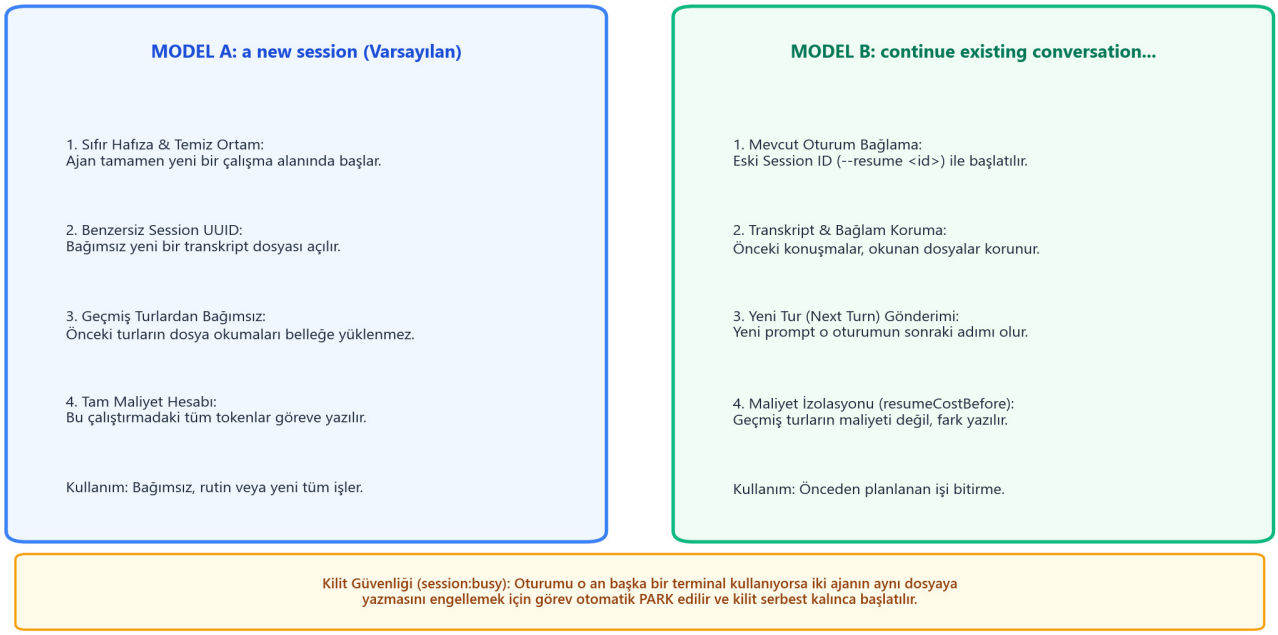
Agent Farm ortamında bir **Görev (FarmTask)**, yapay zeka ajanlarının (Claude Code, Codex, Gemini vb.) belirlenen bir talimatı gözetimsiz (headless) veya etkileşimli (interactive) olarak yürütmesini sağlayan temel çalışma birimidir. Yeni bir görev oluştururken veya mevcut bir görevi düzenlerken karşılaşılan tüm seçenekler, aşağıdaki **3 ana mimari eksen** üzerinde yapılandırılmıştır:

- **1. Başlangıç & Oturum Ekseni (Conversation / S-19):** Görevin ilk çalıştığı anda hangi bağlam ve transkript üzerinde koşacağı (Sıfırdan temiz oturum mu, yoksa var olan bir konuşmanın sonraki turu mu?).
- **2. Zamanlama & Kota Limit Ekseni (When to Retry / M-3, M-10):** Görev başarısız olduğunda veya 5 saatlik kullanım kotasına çarptığında bir sonraki denemenin ne zaman başlayacağı (Açılış saati mi, kademeli dakikalar mı, büyüme eğrileri mi, formül mü?).
- **3. Davranış & Prompt Ekseni (What to Say / M-10, M-12):** Yeniden deneme sırasında sistemin sıfırdan mı başlayacağı yoksa yarım kalan oturumu devam mı ettireceği ve bu sırada hangi promptun iletileceği.

2. Başlangıç Ekseni: Konuşma ve Oturum Yönetimi (S-19)

Formun en üstünde yer alan `Conversation` açılır menüsü, görevin ilk başlatılma anındaki hafıza durumunu belirler:

ŞEMA 1: GÖREV BAŞLANGIÇ OTURUM VE BAĞLAM MODELLERİ (S-19)



Oturum Modellerinin Derinlemesine Analizi

Model	Teknik Çalışma Prensibi	Kullanım Alanı ve Avantajları
`a new session` **VARSAYILAN**	• Tamamen temiz, sıfır hafızalı yeni bir çalışma alanı açar. • Bağımsız yeni bir Session UUID üretilir. • Önceki oturumların dosya okumaları veya bağlamı belleğe yüklenmez. • **Maliyet:** Bu çalıştırmadaki tüm token tüketimi doğrudan bu göreve yazılır.	Yeni, bağımsız ve geçmiş sohbetlerden etkilenmemesi gereken tüm rutin işler, testler ve otomasyonlar için idealdir.
`continue an existing conversation...`	• Geçmişte yapılmış bir oturumun Session ID'sini bağlar (`--resume <id>`).	Console'da ajanla önceden tartışılmış, planlanmış veya yarım bırakılmış bir işi

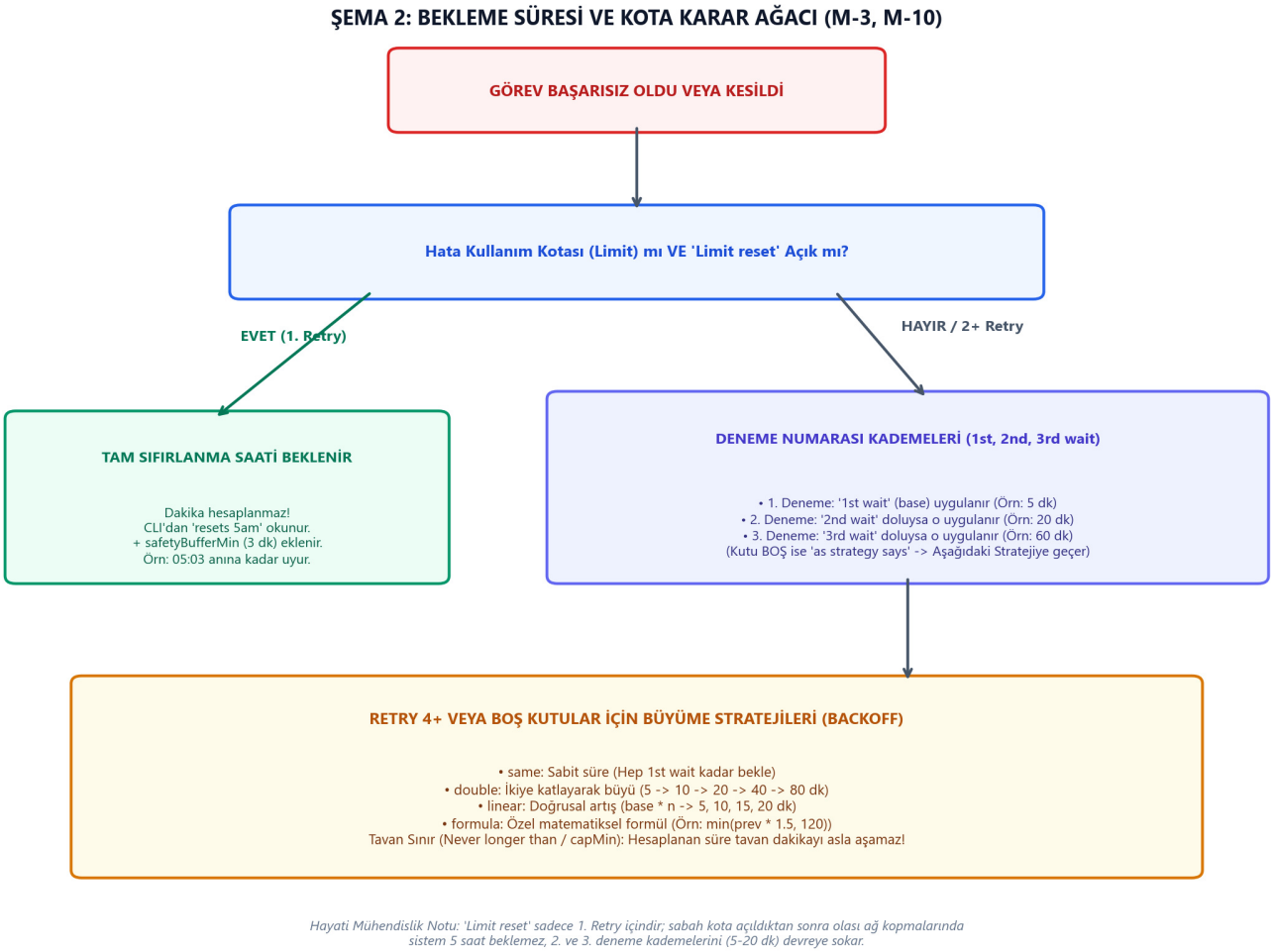
	• O oturumun transkriptini, okuduğu dosyaları ve bağlamını korur. • Yeni prompt, o sohbete **yeni bir tur (next turn)** olarak iletilir. • **Maliyet İzolasyonu:** `resumeCostBefore` sayesinde geçmiş turların maliyeti bu göreve yazılmaz; yalnızca bu turun harcadığı fark faturalandırılır.	arka planda tamamlatmak istediğinizde kullanılır. Bağlam baştan anlatılmak zorunda kalınmaz.

Hayati Mimari Soru: "*Zaten mevcut bir oturumu devam ettiriyorsam, Retry (Yeniden Deneme) kutusunun bir anlamı kalır mı?*"
Cevap: Kesinlikle evet. Çünkü mevcut bir oturuma yeni bir tur gönderdiğiniz an 5 saatlik kullanım kotanız dolmuş olabilir veya oturum sırasında anlık bağlantı kopabilir. Retry kapalı olursa görev anında `failed` olur; açık olursa kota açılış saatini bekleyip mesajı o konuşmaya sabah iletir.

Eşzamanlılık ve Kilit Güvenliği (`session:busy`): Eğer seçilen oturumu o anda Console veya başka bir terminal penceresi kullanıyorsa, sistem iki ajanın aynı transkript dosyasına aynı anda yazıp dosyayı bozmasını engeller. Görev `parked` durumuna alınır ve kilit serbest kalınca otomatik başlatılır.

3. Zamanlama Ekseni: Bekleme Süreleri Nasıl Hesaplanır? (M-3, M-10)

Bir görev başarısız olduğunda veya kotaya çarptığında, bir sonraki denemenin ne zaman başlayacağı **kesin bir öncelik piramidi** ile hesaplanır:



Hesaplama Aşamaları ve Parametreler

Aşama 1: Limit Reset (Kota Takibi) • `resetAware`

- Eğer hata bir kota aşımı (429, Session Limit, Quota Exceeded) ise ve `Limit reset` onay kutusu işaretliyse; sistem dakika tahmini yapmaz.

- CLI çıktısındaki ("*resets 5am*") veya API kullanım verisindeki mutlak zaman damgasını (ISO, Epoch, GMT+3, Europe/Istanbul) okur.
- Üzerine `safetyBufferMin` (varsayılan 3 dakika) ekleyerek görevi tam o ana kadar mutlak uykuya yatırır (`notBefore: 05:03`).

Neden Limit Reset Sadece 1. Retry İçin Geçerlidir? (Kritik Mühendislik Sebebi)
Gece 23:30'da kotaya takılan görev sabah 05:03'e kadar uyutulur (1. Retry). Sabah 05:03'te görev uyandığında **artık 5 saatlik kota DOLU DEĞİLDİR**. Eğer tam o anda anlık bir internet kesintisi veya sunucu hatası (500) olursa, sistemin tekrar 5 saat beklemesi anlamsızdır! İşte bu yüzden 2. ve 3. denemeler için kademeli dakikalar (5 dk, 20 dk vb.) devreye girer.

Aşama 2: İlk Üç Denemenin Özel Kademeleri • `1st, 2nd, 3rd wait`

- **`1st wait`:** İlk denemedeki temel bekleme dakikasıdır. Formüldeki `base` değişkenidir (Zorunlu, varsayılan 40 dk).
- **`2nd wait` & `3rd wait`:** Kademeli bekleme için özel kutulardır (Örn: 5 dk → 20 dk → 60 dk).
- **Boş Bırakma Mantığı ("*as the strategy says*"):** Bu kutular boş bırakılırsa sistem rastgele bir şey yapmaz; doğrudan Aşama 3'teki büyüme stratejisine danışır.

Aşama 3: 4+ Denemeler veya Boş Kutular için Büyüme Stratejisi (Backoff)

- **`same`:** Her denemede 1st wait kadar sabit bekle.
- **`double`:** Süreyi her seferinde ikiye katla (5 → 10 → 20 → 40 → 80 dk).
- **`linear`:** `base × n` doğrusal artış uygula (5, 10, 15, 20 dk).
- **`formula`:** Kullanıcının yazdığı matematiksel formülü çalıştır.
- **`Never longer than (capMin)`:** Süre ne kadar büyürse büyüsün bu tavan dakikayı (örn: 120 dk) asla aşamaz.

Formül (Formula) Motorunun Çalışma Prensibi

Formülde `n` (kaçıncı deneme), `prev` (bir önceki bekleme süresi) ve `base` (1st wait) değişkenleri ile `min(a,b)`, `max(a,b)` fonksiyonları ve `+ - * / ^` operatörleri kullanılır:

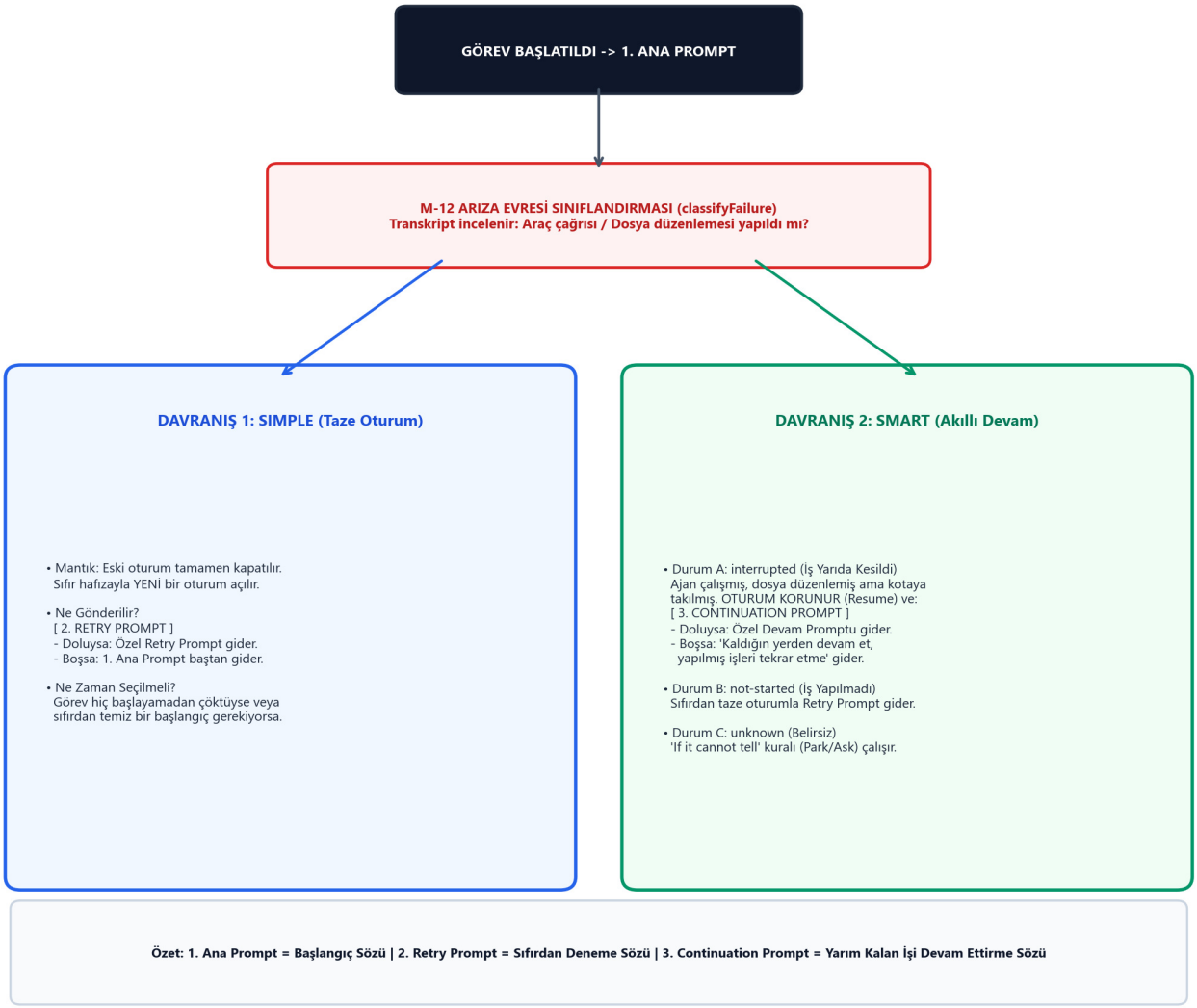
Yazılan Formül	Matematiksel Davranış	Kullanım Amacı
`min(prev * 1.5, 120)`	Süreyi her adımda %50 artırır, 120 dakikada (2 saat) sabitler.	`double` kadar agresif olmayan, yumuşak bir büyüme eğrisi

		istendiğinde.
`min(prev + 15, 60)`	Her denemede bekleme süresine sabit 15 dk ekler, tavanı 60 dk'dır.	Süreyi katlamak yerine sabit adımlarla genişletmek istediğinizde.
`base * (n - 2)^2`	Karesel olarak arayı hızla açar.	İlk denemeleri sık yapıp, başarısızlık uzadıkça arayı hızla açmak için.

4. Davranış & Prompt Ekseni: Ne Söylenecek ve Nerede Söylenecek? (M-10, M-12)

Formda yer alan 3 farklı prompt alanı, görevin yaşam döngüsündeki farklı arıza evrelerinde devreye girer:

ŞEMA 3: ARIZA EVRELERİ VE 3 PROMPT YÖNLENDİRME ALGORİTMASI (M-10, M-12)



M-12 Arıza Evreleri ve Prompt Eşleşmeleri

Arıza Evresi	Nasıl Tespit Edilir?	Sistemin Eylemi ve Kullanılan Prompt
`interrupted` (İş Yarıda Kesildi)	Ajanın transkriptinde en az 1 araç çağrısı veya asistan adımı varsa (yani gerçek iş yapılmışsa).	**Smart Mod:** Oturum kapatılmaz, **AYNI OTURUM DEVAM ETTİRİLİR (Resume)** ve bir sonraki tur olarak `Continuation prompt` gönderilir.
`not-started` (Hiç Başlayamadı)	Ajan 0. saniyede terminal kilitlenmesi veya bağlantı hatasıyla hiç adım atamadan çöktüyse.	Ortada devam edilecek bir bağlam olmadığından sıfırdan yeni oturum açılır ve `Retry prompt` (boşsa Ana Prompt) gönderilir.
`unknown` (Belirsiz)	Transkript diske henüz tam yazılmamışsa veya çıktı okunamıyorsa.	Aşağıdaki "*If it cannot tell*" güvenlik freni devreye girer.

3 Prompt Alanının Görev Ayrımı

- **1.** `Prompt` **(Ana Prompt):** Görevin ilk çalıştığı anda iletilen asıl talimattır (Zorunludur).
- **2.** `Retry prompt`**:** `Simple` modda veya sıfırdan yeni oturum açıldığında orijinal yerine gönderilir. Boşsa **Ana Promptun aynısını** baştan gönderir.
- **3.** `Continuation prompt`**:** `Smart` modda ve görev kotaya takılıp yarıda kaldığında aynı oturuma iletilir. Boşsa standart metni gönderir: "*Carry on from where you stopped. Read the transcript above and the current state of the working folder first, then finish the ORIGINAL task without redoing work that is already done.*"

5. Güvenlik Freni: "If it cannot tell" Nedir?

Bu ayar sadece `What a retry is: smart` seçildiğinde görünür. Bir görev çöktüğünde transkript dosyası henüz diske tam yazılmamışsa (`unknown` evresi), sistem iş yapılıp yapılmadığını kesin olarak bilemez:

ŞEMA 4: 'IF IT CANNOT TELL' BELİRSİZLİK GÜVENLİK FRENİ VE EYLEM MATRİSİ



Ayar Değeri	Sistemin Eylemi	Risk / Güvenlik Analizi
`park` **ÖNERILEN**	Görevi beklemeye alır (`parked`). Arka plan izleyicisi transkripti bağlayıp gerçeği öğrenene kadar bekler; ardından doğru kararı (devam mı, sıfırdan mı) verir.	**En güvenli seçenektir.** Hiçbir kod kaybı veya mükerrer çalışma yaşanmaz.
`simple`	Beklemez; iş yapılmamış varsayarak sıfırdan yeni oturum açar ve promptu gönderir.	**Riskli:** Aslında dosyalar değiştirilmişse, onları görmeden baştan yapmaya çalışabilir.
`ask`	Görevi durdurur ve kullanıcıya bildirim (Notice) göndererek ne yapılacağını sorar.	Kullanıcı tam insan kontrolü istediğinde uygundur.
`stop`	Belirsizlik varsa görevi direkt başarısız (failed) bırakır, tekrar denemez.	Hata toleransı sıfır olan kritik otomasyonlar için.

6. Kullanıcı Arayüzü (UI) Değerlendirmesi ve

Sadeleştirme

Labirentten Çıkış Kılavuzu:

1. Kullanıcıların %95'i için `Retry prompt` ve `Continuation prompt` alanlarını **BOŞ BIRAKMAK** en doğru yoldur. Sistem boş bırakıldığında en kusursuz kararları verir.
2. `Limit reset` kutucuğunu işaretlemek, gece bırakılan tüm uzun görevleri otomatik olarak korur.
3. `If it cannot tell` seçeneğini varsayılan `park` olarak bırakmak en güvenli limandır.

7. Hızlı Karar Matrisi

Uygulamak İstediğiniz Senaryo	Seçilmesi Gereken İdeal Parametreler
1. Gece uzun kodlama / refactor görevi (Kota biterse sabah devam etsin)	• `Conversation: a new session` • `Limit reset:` ☑ İşaretli • `What a retry is: smart` • *Prompt kutuları: BOŞ*
2. Gündüz Console'da konuşulan işi arka planda bitirme	• `Conversation: continue an existing conversation...` (Listeden seç) • `Prompt:` "Tartıştığımız adımları tamamla." • `Limit reset:` ☑ İşaretli
3. Hata olursa sıfırdan farklı bir talimatla deneme	• `What a retry is: simple` • `Retry prompt:` "İlk denemede hata aldın, şimdi şu yöntemle baştan yap..."